

# QA Codebase Walkthrough

Every file added by QA and BA, what each one is for, how it works internally, and what it proves. Written so any team member can read, run, extend or challenge the test suite without a handover conversation.

|                                  |                    |                     |                             |
|----------------------------------|--------------------|---------------------|-----------------------------|
| AUTHOR                           | DATE               | BRANCH              | COMPANION                   |
| Shanthi Subramanian ·<br>BA / QA | 12 September 2026  | qa/ba-qa-workstream | QA & BA Strategy            |
| 13                               | 80                 | 6                   | 0                           |
| FILES ADDED                      | AUTOMATED<br>TESTS | DEFECTS RAISED      | PRODUCTION<br>FILES CHANGED |

## Overview

What was added, where it sits, and how the pieces relate.

THE ONE-LINE SUMMARY

**Nothing under `src/` was modified.** Every file below is new, and lives in `tests/` or `docs/`. QA found six defects without changing a line of production code — the fixes are the developers' to make, and the tests are waiting to confirm them.

## File map

```
tests/
├─ Humaid.RiskGovernance.AdminUI.UnitTests/
│   └─ Assessment/AssessmentServiceTests.cs    NEW 12 tests – the finalization gate
│       └─ README.md                          edited – owner line
├─ e2e/                                         NEW the whole Playwright project
│   ├── package.json package-lock.json .gitignore
│   ├── playwright.config.js                  two projects: api, ui
│   ├── global-setup.js                      aborts if the API is down
│   ├── fixtures/
│   │   ├── users.js                         seeded users by role (API specs)
│   │   └─ app.js                            sign-in, nav, tabs (UI specs)
│   ├── api/
│   │   ├── smoke.spec.js                    3 tests – harness + contract
│   │   └─ authorization.spec.js             9 tests – DEF-001/002/003
│   ├── ui/
│   │   ├── journeys.spec.js                 3 tests – the golden path
│   │   └─ governance.spec.js                3 tests – DEF-004/006
│   ├── tools/make-report-pdf.mjs            regenerates the strategy PDF
│   └─ README.md                             how and why

docs/qa/
├─ qa-ba-strategy.html                       the strategy report, standalone
└─ qa-ba-strategy.pdf                        26 pages, A4
```

## The three suites, and why there are three

| SUITE                               | TESTS | RUNS AGAINST            | REACHES WHAT THE OTHERS CANNOT                                       |
|-------------------------------------|-------|-------------------------|----------------------------------------------------------------------|
| <b>xUnit</b> tests/...<br>UnitTests | 62    | Nothing – all<br>mocked | Service logic in milliseconds, no stack required                     |
| <b>Playwright api</b>               | 12    | API + Postgres          | HTTP status mapping and authorization, invisible to a<br>mocked test |
| <b>Playwright ui</b>                | 6     | Full stack +<br>browser | Whether the screens actually compose into a working<br>workflow      |

**80 tests total.** Still unreachable by all three: the `audit_event` append-only trigger and the `CHECK` constraints, because no HTTP endpoint attempts an `UPDATE`. That needs a Testcontainers suite and remains the top gap.

## Running everything

```
# 1. stack
bash ops/setup-local-db.sh                # Postgres + Azurite
dotnet run --project src/6-MockExternalSystems/Humaid.RiskGovernance.MockSystems
dotnet run --project src/1-API/Humaid.RiskGovernance.AdminUI.Web
cd webapp && npm run dev                    # :3000, needed by the ui project only

# 2. tests
dotnet test tests/Humaid.RiskGovernance.AdminUI.UnitTests  # 62 passed
cd tests/e2e && npm install && npm test                    # 18 passed
npm run report                                             # HTML report + traces
```

# Unit tests

The only file added to the existing xUnit project.

## tests/Humaid.RiskGovernance.AdminUI.UnitTests/Assessment/AssessmentServiceTests.cs

295 LINES

12 TESTS

XUNIT + MOQ

US-6.3

### Why it exists

`AssessmentService.FinalizeAsync` is the single chokepoint enforcing CLAUDE.md's central rule. `CommitteeService.CastVoteAsync` refuses a vote unless the request is already `PendingCommittee`, so finalization is the *only* route to a committee decision. It had **no test at all**. That made it the highest-value thing in the repo to cover.

### Internals

#### 1. Type aliases, to dodge a real C# collision

The model class and its namespace share a name (`...Models.CategoryMapping.CategoryMapping`), which makes an unqualified reference ambiguous. The alias keeps the test body readable:

```
using CategoryMappingRecord =  
    Humaid.RiskGovernance.AdminUI.Infrastructure.Models.CategoryMapping.CategoryMapping;
```

#### 2. Four mocked repos, one system under test

```
private readonly Mock<IAssessmentRepo> _assessmentRepo = new();  
private readonly Mock<ICategoryMappingRepo> _categoryMappingRepo = new();  
private readonly Mock<INarrativeSectionRepo> _narrativeSectionRepo = new();  
private readonly Mock<IPolicyResearchRepo> _policyResearchRepo = new();  
private readonly AssessmentService _sut;
```

#### 3. A `Given(...)` helper that makes each test one readable sentence

Readiness depends on three collections agreeing with each other. Setting all three inline in every test would bury the point, so one helper arranges the world and each test states only what it is varying:

```
private void Given(  
    IReadOnlyList<CategoryMappingRecord> categories,  
    IReadOnlyList<NarrativeSection> sections,  
    IReadOnlyList<PolicyReliance> reliance)  
{  
    _categoryMappingRepo.Setup(r => r.GetMappingAsync(AssessmentId)).ReturnsAsync(catego  
    _narrativeSectionRepo.Setup(r => r.GetSectionsAsync(AssessmentId)).ReturnsAsync(sect  
    _policyResearchRepo.Setup(r => r.GetRelianceAsync(AssessmentId)).ReturnsAsync(relian  
}
```

Plus three small builders — `Category(...)`, `Section(...)`, `Reliance(...)` — so a test reads as domain language rather than object construction.

## The twelve tests

| TEST                                                                                   | PROVES                                                                                  |
|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <code>FinalizeAsync_BlocksAndNamesTheCategoryWhenItsNarrativeIsStillAiDrafted</code>   | US-5.1 AC2 — an AI draft cannot route to committee                                      |
| <code>FinalizeAsync_BlocksWhenAMappedCategoryHasNoNarrativeSectionAtAll</code>         | Absence ≠ "nothing outstanding"                                                         |
| <code>FinalizeAsync_BlocksWhenAMappedCategoryHasNoPolicyRelianceDecision</code>        | US-3.2 AC2                                                                              |
| <code>FinalizeAsync_NeverFinalizesTheAssessmentWhenBlocked</code>                      | The repo is never reached — throwing <i>after</i> persisting would still let it through |
| <code>FinalizeAsync_ListsEveryOutstandingReasonNotJustTheFirst</code>                  | US-6.3 AC1 — one message lists everything                                               |
| <code>FinalizeAsync_RecordsTheFinalizingAnalystWhenEveryGateIsSatisfied</code>         | US-6.3 AC3 — never anonymous                                                            |
| <code>CheckReadinessAsync_TreatsAReviewedOrEditedSectionAsSatisfied</code>             | US-6.3 AC2 (a <code>[Theory]</code> , 2 cases)                                          |
| <code>CheckReadinessAsync_IgnoresACategoryTheAnalystDeactivated</code>                 | A removed category must not block forever                                               |
| <code>CheckReadinessAsync_AcceptsAnAssessmentWideRelianceForEveryMappedCategory</code> | The <code>RiskCategoryI == null</code> branch                                           |
| <code>CheckReadinessAsync_ReportsReadyWhenNoCategoriesAreMappedAtAll</code>            | <b>DEF-004</b> — characterization see below                                             |
| <code>OpenWorkspaceAsync_DelegatesToTheRepo</code>                                     | Delegation                                                                              |

#### THE CHARACTERIZATION TEST – READ THIS ONE CAREFULLY

The last readiness test *asserts behaviour that is arguably wrong*, and says so in its own comment. With zero active categories both readiness loops are no-ops, so `IsReady` is vacuously true. Rather than quietly "fixing" someone else's module, QA pinned the current behaviour so that any future change to it is deliberate and visible — and raised it as DEF-004. It was later confirmed end to end through the UI.

### How we know these tests actually work

A green suite is a claim, not evidence. The gate condition was temporarily replaced with `if (false)` — effectively deleting the rule — and the suite re-run:

```
Failed: 5, Passed: 57, Total: 62
```

Exactly the five `FinalizeAsync_*` tests failed. The other seven stayed green because they call `CheckReadinessAsync` directly and never pass through the gate. That selectivity is the signal: a blunt suite would have failed more, or fewer, and told us nothing. Production code was then restored byte-for-byte (`git diff -- src/` empty).

---

# The Playwright harness

Configuration, safety guard, and the two fixture modules everything else builds on.

tests/e2e/playwright.config.js

47 LINES

2 PROJECTS

## Purpose

Defines two independent projects so the cheap suite does not inherit the expensive one's dependencies.

`api` needs only the API; `ui` additionally needs the Vite dev server.

```
projects: [  
  { name: 'api', testDir: './api', use: { baseURL: API_BASE_URL,  
    extraHTTPHeaders: { Accept: 'application/json' } } },  
  { name: 'ui', testDir: './ui', use: { baseURL: WEBAPP_BASE_URL,  
    trace: 'retain-on-failure', screenshot: 'only-on-failure' } },  
]
```

## Decisions worth knowing

- `forbidOnly: !!process.env.CI` — a stray `.only` silently narrows the suite. Fine locally, never in CI.
- `retries: CI ? 1 : 0` — no retries locally, so flakiness is visible rather than masked.
- `trace: 'retain-on-failure'` on `ui` only — a trace is the difference between "step 6 failed" and seeing exactly what was on screen. Too heavy for the API project, which has nothing visual to show.
- Base URLs come from `API_BASE_URL` / `WEBAPP_BASE_URL` so CI overrides config without editing the file.

## tests/e2e/global-setup.js

29 LINES

LOAD-BEARING

### Purpose — and why it is not merely politeness

Pings `/api/Ping` before any test runs and aborts the whole run with setup instructions if it fails.

#### THE REASON THIS FILE EXISTS

Several tests use `test.fail()` to record known defects. A `test.fail()` test "passes" when it fails for *any* reason — including a refused connection. Without this guard, running the suite with nothing listening on :5210 would produce a fully green board that proves absolutely nothing. The guard converts that silent lie into a loud, actionable error.

Verified: with the stack down the run exits 255 and prints the three commands needed to start it.

`/api/Ping` is the right probe because `PingController` deliberately carries no `[Authorize]`.

## tests/e2e/fixtures/users.js

37 LINES

API SPECS

### Purpose

Resolves the six seeded users **by role, never by hardcoded id** — ids are generated by `seed_dev_users.sql`, so pinning them would break on every re-seed.

```
const byRole = (role) => users.find((user) => (user.role ?? user.Role) === role);
return { all: users, productOwner: byRole('ProductOwner'), analyst: byRole('Analyst'),
  committeeMember: byRole('CommitteeMember'), admin: byRole('Admin') };
```

It throws a diagnostic error rather than returning empty when the table looks unseeded — so "the DB was not set up" never masquerades as "the endpoint returned nothing".

Also exports `NON_EXISTENT_ID`: a well-formed GUID that deliberately matches no record, used by probes that must be rejected on *authorization* grounds before any lookup happens.

## Purpose

Everything the browser journeys need: signing in as a role, two selector helpers, and an API shortcut for arranging preconditions.

### `openAppAs(browser, role)` — there is no login to automate

While Auth0 is unconfigured, `DevUserContext.jsx` reads the acting user straight out of `localStorage.devUserId`. Seeding that key before first paint *is* the sign-in — faster and far less brittle than driving a user-picker, and it uses the app's own mechanism rather than a test-only backdoor.

```
await context.addInitScript((id) => window.localStorage.setItem('devUserId', id), user.i
await page.goto(WEBAPP_BASE_URL, { waitUntil: 'domcontentloaded' });

const bypass = page.getByRole('button', { name: /Continue without signing in/i });
if (await bypass.count()) await bypass.click();

// Proves the role took effect before any assertion depends on it.
await expect(page.getByText(`${user.displayName} - ${role}`)).toBeVisible();
```

That final assertion matters: without it, a failure to switch user would surface later as a confusing "element not found" in whatever screen the test expected.

### `nav(page, label)` — a one-line fix for a real trap

```
export function nav(page, label) {
  return page.getByRole('button', { name: label, exact: true });
}
```

The sidebar has **"Submit Request"**; the intake form has **"Submit request"**. Playwright's accessible-name matching is case-insensitive unless `exact` is set, so without this *every* nav click is a strict-mode violation resolving to two elements. This cost real debugging time and is exactly the kind of thing worth centralising.

### `createChangeRequest(title)` — arrange via API, assert via UI

Returns the created record ( `{ id, requestNumber, ... }` ). Tests whose subject is a *later* stage use this instead of re-driving intake through the browser: faster, independent of journey ordering, and it keeps each test's failure pointing at the thing it actually tests.

# API specs

Twelve tests at the HTTP boundary — where authorization actually lives.

tests/e2e/api/smoke.spec.js

51 LINES3 TESTS

### Purpose

Proves the harness itself works and pins the API's contract. **If these fail, the problem is the stack — not the thing any other spec is measuring.** That separation is the whole point of the file.

| TEST                                                          | PINS                                                                                                                                                                                                                  |
|---------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| every response uses the <code>OperationResult</code> envelope | <code>BaseApiController.ExecuteAsync</code> wraps every action, so <code>data</code> — not a bare body — is the contract both the webapp and these tests rely on                                                      |
| the seeded users cover all four roles                         | Six users across <code>ProductOwner</code> / <code>Analyst</code> / <code>CommitteeMember</code> / <code>Admin</code> . Every role journey depends on this                                                            |
| the FFIEC risk framework is seeded with its four categories   | <code>PRODUCTS_SERVICES</code> , <code>CUSTOMERS_ENTITIES</code> , <code>GEOGRAPHIC_LOCATIONS</code> , <code>DELIVERY_CHANNELS</code> — a partial seed would silently weaken every category-mapping result downstream |

## tests/e2e/api/authorization.spec.js

185 LINES

9 TESTS

DEF-001

DEF-002

DEF-003

### Purpose

Records the access-control findings. Structured as four `describe` blocks so the file reads as an argument rather than a list.

| BLOCK                               | TESTS | WHAT IT ESTABLISHES                                                                                                                       |
|-------------------------------------|-------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Authentication</b>               | 2     | The user directory is readable with no credential ( <b>FAILS</b> ), and <code>/api/Ping</code> is open <i>by design</i> ( <b>PASSES</b> ) |
| <b>Role enforcement</b>             | 3     | A Product Owner is not refused when voting, finalizing or changing scoring config ( <b>ALL FAIL</b> )                                     |
| <b>Input validation still holds</b> | 2     | A mitigation factor of <code>1.0</code> is refused; a malformed GUID 404s ( <b>BOTH PASS</b> )                                            |
| <b>Error handling</b>               | 2     | Unknown ids surface as 500 rather than 4xx ( <b>FAILS</b> )                                                                               |

#### WHY TWO PASSING TESTS SIT IN A DEFECT FILE

The "Input validation still holds" block is deliberate and load-bearing. It proves the API is *not* simply accepting everything — so the finding stays precisely an **authorization** gap and is never overstated into "the API is wide open". Keeping the counter-evidence in the same file is what makes the rest of it credible.

### The `test.fail()` pattern

Every defect test asserts the **secure** behaviour and is annotated `test.fail()`:

```
test('the user directory is not readable without a credential', async ({ request }) => {
  test.fail(); // DevBypassAuthHandler authenticates every request; returns 200 + all si

  const response = await request.get('/api/User');
  test.info().annotations.push({ type: 'actual status', description: String(response.sta

  expect(response.status(), 'unauthenticated read of /api/User should be rejected').toBe
});
```

Three things happen at once. The board stays green while the gap is open. The moment somebody implements role checks, Playwright reports *"expected to fail, but passed"* — **the fix announces itself**. And the annotation records the *actual* status in the HTML report, so the evidence travels with the test.

**When a defect is fixed, delete the annotation — not the test.**

## Verified results

| PROBE                                    | EXPECTED | ACTUAL     |
|------------------------------------------|----------|------------|
| GET /api/User – no credential            | 401      | <b>200</b> |
| POST /api/Committee/Vote as PO           | 403      | 400        |
| POST /api/Assessment/{id}/Finalize as PO | 403      | 500        |
| POST /api/Scoring/Config as PO           | 403      | 500        |

**No endpoint returned 403.** Stated honestly in the file's own comments: the last three used a non-existent id, so each was rejected on *data* grounds before role could have mattered — they prove the role check does not run *first*, not that a valid request would be accepted.

---

# UI specs

Six browser tests: the golden path, and the two defects only the UI could reveal.

## tests/e2e/ui/journeys.spec.js

139 LINES   3 TESTS   SERIAL

### Purpose

The golden path as **one continuous story**, not three unrelated fixtures: one change request moving through the workflow, exactly as an examiner would reconstruct it from the audit trail.

```
test.describe.configure({ mode: 'serial' });

const change = { title: `Launch same-day international wire transfer service ${Date.now()}`,
  requestNumber: null };
```

Serial is the right call here: journey 2 has nothing to assess until journey 1 has submitted something. If journey 1 fails, 2 and 3 skip rather than producing three confusing failures with one cause. The timestamped title keeps runs independent as the database accumulates data.

### What each journey asserts

| JOURNEY                   | STEPS                                                                                        | ACCEPTANCE CRITERIA                                                                                                                                   |
|---------------------------|----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1 – Product Owner</b>  | Pick change type from the combobox, fill title / description / type-specific details, submit | US-1.1 AC4 — a unique immutable <code>CR-2026-00001</code> is assigned<br>US-1.3 — status visible without emailing FCRM                               |
| <b>2 – FCRM Analyst</b>   | Find it in the inbox, open the workspace, walk all seven tabs, finalize, route               | US-1.1 — reaches the analyst inbox<br>US-9.1 — intake is already on the audit trail, attributed to a named human, before the analyst touches anything |
| <b>3 – Risk Committee</b> | Find it in the queue, open, cast an Approve vote                                             | US-8.1 — a finalized assessment reaches the queue<br>US-8.2 AC5 — the vote is recorded against the member <i>by name</i> , never anonymised           |

#### ONE DELIBERATE WORKAROUND, OPENLY MARKED

Journey 2 retries the Open click, because DEF-006 makes the first attempt fail. That is what an analyst does today, so the journey does it too — but the defect is asserted *separately* in `governance.spec.js` rather than being silently absorbed here. A journey that quietly works around a bug, with no test naming it, is how bugs become permanent.

## Selector notes, learned the hard way

| THING               | WHAT WORKS, AND WHY                                                                                                                                                          |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Nav vs form buttons | <code>exact: true</code> is <b>mandatory</b> — "Submit Request" and "Submit request" differ only by case                                                                     |
| Change type         | Not a <code>&lt;select&gt;</code> — a base-ui combobox. Scope to <code>page.locator('form').first()</code> , because the header carries a third combobox (the user switcher) |
| Intake fields       | Stable ids <code>#title</code> , <code>#description</code> , <code>#details</code> , <code>#file</code> . Inputs have no <code>name</code> attributes                        |
| Workspace stages    | <code>role="tab"</code> , not buttons: Categories, Policy, Extraction, Narrative, Scoring, Finalize, Audit                                                                   |

## DEF-004 — an empty assessment reaches the committee

**The most serious finding in the product so far**, and the clearest example of the unit suite and the UI suite working together: it was *predicted* by a characterization unit test, then *confirmed* in the browser.

`CheckReadinessAsync` iterates over the active mapped categories. With none mapped, both loops are no-ops and `IsReady` is vacuously true — so the Finalize tab reports **"Ready to finalize."**

Verified end to end on 2026-09-12: a change request with **zero categories, zero policy reliance decisions, zero extracted fields, zero narrative and zero risk scores** was finalized, routed, and appeared in the committee queue with a live **"Submit vote"** button. The committee is being asked to take an accountable decision on a document containing nothing but a title.

Two tests cover it: one asserts the Finalize tab refuses it; one asserts it never reaches the queue.

## DEF-006 — opening a workspace fails the first time, every time

`func_getOrCreateAssessment` does SELECT-then-INSERT with no `ON CONFLICT` and no lock:

```
SELECT id INTO v_id FROM assessment WHERE change_request_id = p_change_request_id;
IF v_id IS NULL THEN INSERT INTO assessment ...
```

React 18 StrictMode double-invokes effects in dev, so two POSTs race. Both read `NULL`, both insert, and the loser violates `UNIQUE (change_request_id)` → Npgsql 23505 → **HTTP 500**. The row *is* created, so a second click works.

### A TEST THAT WAS WRITTEN, THEN DELIBERATELY DELETED

A browser-level version — click Open, assert no 500 — was written first and **removed**. It only fails when the two requests genuinely overlap, and it passed on one full-suite run while failing in isolation.

**A `test.fail()` marker that oscillates is worse than no test, because it trains everyone to ignore the signal.**

The kept version drives the race directly with `Promise.all`, reproduces 23505 every time, and makes the stronger claim anyway: this is a stored-function defect, not a React dev-mode artefact. Two analysts opening the same request simultaneously would hit it in production, where StrictMode does not apply.

```
const open = () => fetch(`${API_BASE_URL}/api/Assessment/OpenWorkspace/${change.id}`,
  { method: 'POST' });
const [first, second] = await Promise.all([open(), open()]);

expect([first.status, second.status],
  'func_getOrCreateAssessment must be idempotent under concurrent callers').toEqual([200
```

Fix for both halves: `INSERT ... ON CONFLICT (change_request_id) DO NOTHING`, then re-select.

---

## Tooling & documentation

The three non-test artefacts.

### tests/e2e/tools/make-report-pdf.mjs

57 LINES

REPRODUCIBILITY

#### Purpose

Regenerates the strategy report's standalone HTML and PDF from one source, using the Chromium already installed for Playwright — so the PDF never becomes a stale binary nobody can rebuild.

```
node tools/make-report-pdf.mjs <src.html> <out.pdf> <out.html>
```

#### Two details that matter

- **It wraps the fragment.** The artifact source has no `<html>/<head>` — the Artifact runtime supplies that at publish time. A standalone file must carry its own, or it misrenders when opened from disk or attached to an email.
- **It waits on `document.fonts.ready`.** Without this the export silently falls back off IBM Plex and the entire type system is lost — a failure you would only notice by comparing side by side.

### tests/e2e/README.md

201 LINES

#### Purpose

The in-repo entry point for anyone who opens `tests/e2e` and asks "what is this and how do I run it". Covers: why there are two test stacks and which layer each belongs to; how to run each project; how to read `test.fail()` results; both findings with verified status codes; the selector traps; and the layout.

It deliberately states **what the suite does not prove** — the scoping caveat on DEF-002, and the fact that no HTTP test can ever reach the `audit_event` trigger. A README that only lists strengths is a sales document, not a handover.

## Purpose

The companion strategy report — nine sections: the problem statement, requirement analysis, the solution, the testing this project needs, BA approach, testing approach, automation strategy, work completed, and the defect register with runnable reproduction steps and a five-minute demo script.

Published in three forms from one source — a live link, standalone HTML, and A4 PDF — so they cannot drift. A print stylesheet forces the light palette (a PDF has no viewer theme), keeps `print-color-adjust: exact` so severity colours survive export, and holds findings and tables together across page breaks.

---

## Why it is built this way

Five techniques used throughout, each solving a specific failure mode. Worth understanding before extending the suite.

### 1 · `test.fail()` for known defects

Assert the *correct* behaviour, mark the test expected-to-fail. The board stays green while the gap is open; when the fix lands, Playwright reports "expected to fail, but passed" and the fix announces itself. The alternative — deleting the test, or weakening it to match the bug — loses the requirement permanently.

### 2 · Characterization tests for behaviour that is not ours to change

Where current behaviour is arguably wrong but the fix belongs to another module's owner, pin it with a clearly-commented test. Future change becomes deliberate and visible rather than accidental. DEF-004 was found exactly this way.

### 3 · Mutation checks, because green means nothing on its own

Break the rule deliberately, confirm the right tests fail, revert. Done for the finalize gate: five failures, precisely the five guarding it. This is the single most useful thing to show a reviewer who asks whether a green suite means anything.

### 4 · A guard against false greens

`global-setup.js` exists solely because `test.fail()` and a dead server are indistinguishable without it. Any suite mixing expected-failures with real infrastructure needs this, or it will eventually report success while testing nothing.

### 5 · Counter-evidence kept alongside findings

Passing validation tests sit inside the defect file; the "Ping stays deliberately open" test sits inside the authentication block. Both exist so the findings stay precise. Scope claims honestly — the DEF-002 caveat is written into the test file, the README and the report, because a finding that overstates itself gets argued down and takes the true part with it.

#### IF YOU EXTEND THIS SUITE, KEEP THESE

Prefer edge cases over golden-path duplicates. Keep the UI suite at three journeys. Name the user story every test proves, in the test's own comment — that is what makes a traceability matrix mechanical rather than a separate invention. And never let a test pass for a reason you have not checked.

# Handover

Current state, what is owned by whom, and what comes next.

## Defect register

| ID      | FINDING                                                               | SEVERITY | PROVEN BY            | OWNER        |
|---------|-----------------------------------------------------------------------|----------|----------------------|--------------|
| DEF-001 | User directory readable with no credential                            | HIGH     | api/authorization    | Suleman      |
| DEF-002 | No role enforcement; actor is self-declared in the body               | HIGH     | api/authorization    | Suleman      |
| DEF-003 | Unknown ids surface as HTTP 500                                       | MEDIUM   | api/authorization    | Suleman      |
| DEF-004 | Empty assessment finalizes and reaches the committee                  | HIGH     | unit + ui/governance | Suleman      |
| DEF-005 | <code>control</code> table never seeded — scores cannot be explained  | MEDIUM   | manual + API         | Suleman + BA |
| DEF-006 | <code>func_getOrCreateAssessment</code> race — first open always 500s | MEDIUM   | ui/governance        | Suleman      |

## Next, in priority order

1. **Testcontainers suite** — the `audit_event` append-only trigger and the `CHECK` constraints are the repo's flagship governance claims and are *unreachable* by all 80 current tests. Needs no API key; Docker is already a prerequisite.
2. **App CI pipeline** — both existing Azure DevOps pipelines are path-filtered to `iac/environments/dev/*`, so nothing in CI runs `dotnet test` or Playwright. Testing is one of six graded SDLC stages.
3. **Adversarial eval cases** — 10 of 12 cases are still skipped for want of a configured model; the evaluation framework is 10% of the score.
4. **BA documents** — RBAC matrix (now backed by DEF-002 evidence), non-functional requirements, traceability matrix.

### TWO THINGS TO RAISE WITH THE TEAM

**Suleman** — the six defects above, plus decisions on open questions Q3 (does a scoring override need secondary sign-off?) and Q5 (who approves configuration changes?).

**Suraj** — `release/1.00` still needs merging into `main` before application CI is possible at all; and `ops/setup-local-db.sh` assumes `psql` on the host, which breaks a fresh clone on a Docker-only machine.

---

All test results, status codes and defect reproductions in this document were observed against a running local stack on 12 September 2026 — Postgres and Azurite in Docker, Mock Systems on :5220, the API on :5210 and the Vite dev server on :3000. Where a claim is unverified or scoped, it is marked as such. Companion document: *QA & BA Strategy*.